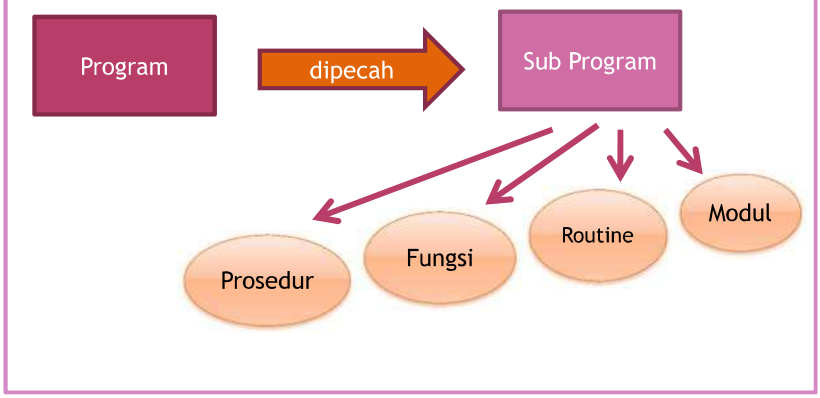
FUNGSI DALAM BAHASA PEMROGRAMAN C

Jurusan Teknik Elektro
Program Studi Teknik Telekomunikasi
A. Haidar, MT.
2022

AGENDA

- Definisi
- Nilai Akhir (Return)
- Function Tanpa Nilai Akhir
- Prototipe Fungsi
- Parameter Formal dan Aktual
- Cara Melewatkan Parameter

DEFINISI



DEFINISI (CONT.)

- Fungsi merupakan bagian (blok) dari kode program yang dirancang untuk melaksanakan tugas tertentu dengan tujuan:
  - Program menjadi terstruktur, sehingga mudah dipahami dan mudah dikembangkan.
  - Dapat mengurangi pengulangan kode (duplikasi kode).

DEFINISI (CONT.)

- Tugas khusus yang dilakukan suatu fungsi dapat dilihat dari beberapa fungsi standard, misalnya:
  - getch( ), fungsi untuk membaca kode tombol papan ketik yang ditekan.
  - printf( ), fungsi untuk menampilkan informasi atau data ke layar.

DEFINISI (CONT.)

- Pada umumnya fungsi memerlukan masukan yang dinamakan sebagai **argumen atau parameter.**
- Masukan ini selanjutnya diolah oleh fungsi dan hasil keluarannya dapat berupa sebuah nilai (disebut nilai keluaran fungsi atau nilai balik fungsi).

DEFINISI (CONT.)

- Bentuk struktur umum fungsi :

PenentuTipe **NamaFungsi**(*Paramter1, ..., Parameter_ke-N*)
{

tubuh fungsi **atau** ***isi fungsi***

}

DEFINISI (CONT.)

- Penentu tipe digunakan untuk menentukan tipe keluaran fungsi, yang dapat dilipih dari salah satu tipe data yang berlaku dalam C, seperti *char* atau *int* dan lainnya.
- Jika penentu tipe tidak disebutkan, maka hasil keluaran akan dianggap sebagai *int*.
- Sebuah fungsi dapat saja tidak mengandung parameter sama sekali, bahkan tanpa deklarasi parameter

DEFINISI (CONT.)

- Sebuah fungsi dapat saja tidak mengandung parameter sama sekali, bahkan tanpa deklarasi parameter
- Contoh fungsi tanpa parameter

```
                          Nama funsi
                          Tanpa argumen
Inisialisasi()    <---    tidak ada tanda titik-koma
{                 <---    awal fungsi
    return(0);    <---    tubuh fungsi
}                 <---    akhir fungsi
```

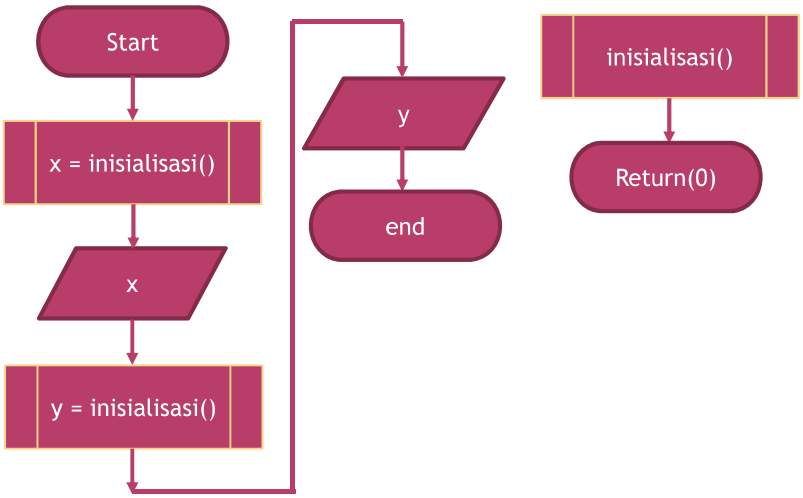
ALGORITMA FUNGSI

- Bentuk algoritma fungsi berdasarkan konsep flowchart adalah sebagai berikut:



- Gambar di atas menggambarkan sebuah fungsi dengan nama *inisialisasi* **yang tidak memiliki parameter serta merupakan fungsi yang menghasilkan nilai akhir (***return) bertipe integer, karena diawal nama fungsi tidak didefinisikan tipe nilai akhir-nya*

IMPLEMENTASI FUNGSI PADA FLOWCHART



NILAI AKHIR (RETURN)

Perhatikan contoh Program 1 berikut !

```
#include <stdio.h>
    inisialisasi();
    main()
    {
            int x, y;
            x = inisialisasi();
            printf("x = %d\n", x);
            y = inisialisasi ();
            printf("y = %d\n", y);
            return(0);
    }

    inisialisasi()
    {
            return(0);
    }
```

Contoh hasil eksekusi:
C>Program_1
x = 0
y = 0

NILAI AKHIR (RETURN) - CONT.

- Sebuah fungsi dapat mempunyai nilai keluaran dengan cara memberikan nilai keluaran pada *variable* yang terletak setelah pernyataan *return* atau menyisipkan nilai secara langsung setelah pernyataan *return*.
- Contoh pada program fungsi menjelaskan bahwa suatu fungsi cukup didefinisikan satu kali, tetapi dapat digunakan beberapa kali untuk menghindari duplikasi kode dan menghemat penulisan program maupun penyimpanan kode dalam memori.

NILAI AKHIR (RETURN) - CONT.

- Bagi suatu fungsi, jika suatu pernyataan return dieksekusi, maka eksekusi terhadap fungsi akan berakhir dan nilai pada parameter *return* akan menjadi keluaran fungsi.
- Untuk fungsi yang tidak memiliki pernyataan return, maka diakhir pernyataan *return* akan disisipkan nilai “0” (angka nol)seperti contoh sebelumnya.
- Pada contoh berikut, fungsi digunakan untuk memperoleh nilai minimum diantara dua nilai yang ada pada parameternya.

NILAI AKHIR (RETURN) - CONT.

```
1.  #include <stdio.h>
2.  minimum(int x,int y)
3.  {
4.       if (x < y)
5.          return(x);
6.       else
7.          return(y);
8.  }
9.  main()
10. {
11.      int a = 20;
12.      int b = 44;
13.      int hasil;
14.      hasil = minimum(a,b);
15.      printf("Nilai terkecil = %d\n", hasil);
16.      printf("Nilai terkecil = %d\n", minimum(3,2));
17. }
```

contoh eksekusi **Program 2** :

C>Program_2
Nilai terkecil = 20
Nilai terkecil = 2

NILAI AKHIR (RETURN) - CONT.

Hal penting berkaitan contoh program sebelumnya:

- Fungsi tsb mempunyai dua buah parameter berupa x dan y yang dideklarasikan dengan jenis tipe data berupa *integer*.

- Penentuan nilai keluaran fungsi dilakukan pada tubuh fungsi berupa pernyataan:

```
if (x < y)
  return(x); → nilai keluaran fungsi adalah nilai x
else
  return(y); → nilai keluaran fungsi adalah nilai y
```

NILAI AKHIR (RETURN) - CONT.

Pada contoh program tsb, pemanggilan fungsi dilakukan sebanyak **dua kali.**

- **pemanggilan pertama:**

  hasil = minimum(a,b);

  memberikan hasil berupa nilai terkecil diantara a dan b untuk disimpan pada variabel dengan nama *hasil*.

- **Pemanggilan kedua:**

  printf("Nilai terkecil = %d\n", minimum(3,2));

  Mencetak hasil pemanggilan fungsi ke layar monitor, dengan *output berupa:*

  Nilai terkecil = 2

FUNGSI TANPA NILAI *RETURN*

- Penggunaan fungsi dapat dilakukan tanpa memberikan nilai akhir (return), yaitu dengan menambahkan pernyataan *void* di awal nama fungsi. seperti contoh berikut :

```
void function_name (tipe_data parameter)
{
    ......
}
```

FUNGSI TANPA NILAI *RETURN* (CONTOH KE-1)

```
1.  # include <stdio.h>
2.  void bahasa ( )
3.  {
4.      printf ("bahasa Indonesia\n");
5.  }
6.
15. void language ( )
16. {
17.     printf ("English Language");
18. }

19. main()
20. {
21.     bahasa ( );
22.     language ( );
23. }
```

hasil eksekusi pada File Program3 :

bahasa Indonesia
English Language

FUNGSI TANPA NILAI *RETURN* (CONTOH KE-2)

```
1.  # include <stdio.h>
2.  void bahasa ( );
3.  void language ( );
4.  main()
5.  {
6.      bahasa ( );
7.      language ( );
8.  }
9.
10. void bahasa ( )
11. {
12.     printf ("bahasa Indonesia\n");
13. }
14.
15. Void language ( )
16. {
17.     printf ("English Language");
18. }
```

hasil eksekusi pada Program4 :

bahasa Indonesia
English Language

PROTOTIPE FUNGSI

- Dari kedua contoh program sebelumnya, dapat dilihat bahwa penulisan fungsi dapat diletakkan pada awal program maupun pada akhir program.
- Pada contoh ke-1, pendefinisian fungsi dilakukan di awal program sebelum pernyataan program utama, fungsi main(). Model penulisan ini merupakan **model klasik**.
- Pada contoh ke-2, pendefinisian fungsi dilakukan setelah program utama, namun diawal program sebelum program utama, fungsi main(), harus dituliskan nama fungsi dan parameternya yang diakhiri dengan tanda **titik-koma**. Model ini dikenal dengan nama *Fungsi Prototipe.*

PROTOTIPE FUNGSI (CONT.)

- Pendefinisian fungsi diawal digunakan untuk menjelaskan kepada kompiler mengenai:
  - Tipe keluaran fungsi, nilai *return;*
  - Jumlah parameter;
  - Tipe dari masing-masing parameter.
- Bagi kompiler, informasi yang didefinisikan diawal fungsi digunakan untuk memeriksa keabsahan (validation code) sekaligus proses koreksi pada parameter dalam pemanggilan fungsi serta penamaan dan tipe data fungsi.

PROTOTIPE FUNGSI (CONT.)

Perhatikan contoh Program 3 berikut :

```
1.  #include <stdio.h>
2.  float jumlah(float x,float y);
3.  main()
4.  {
5.      int a = 6;
6.      int b = 3;
7.      float c;
8.      c = jumlah(a,b);
9.      printf("a + b = %g\n",c);
10.     printf("Hasil penjumlahan = %g\n", jumlah(20.1, 0.9));
11. }
12. float jumlah(float x, float y)
13. {
14.     return(x + y);
15. }
```

Prototipe Fungsi

hasil eksekusi :
c>Program_3
a + b = 9
Hasil penjumlahan = 21

PARAMTER FORMAL DAN AKTUAL

- Parameter formal adalah variabel yang ada pada daftar parameter dalam definisi fungsi. Pada fungsi jumlah( ) Program_3, parameter x dan y dinamakan sebagai parameter formal.
- Parameter aktual adalah parameter (tidak selamanya menyatakan variabel) yang digunakan dalam pemanggilan fungsi. Pada pernyataan:

  c = jumlah(a, b);

  y = jumlah(20.1, 0.9);
- Parameter a dan b merupakan parameter aktual dari fungsi jumlah (), demikian pula 20.1 dan 0.9 (meskipun bukan berupa variabel).

CARA MELEWATKAN PARAMETER

- Terdapat dua cara untuk melewatkan parameter ke dalam fungsi, yaitu dengan cara:
  - Pemanggilan dengan nilai (call by value)
  - Pemanggilan dengan referensi (call by reference)
- Pemanggilan dengan nilai telah dijelaskan pada contoh program sebelumnya.

CARA MELEWATKAN PARAMETER

- Pada pemanggilan dengan nilai, nilai dari parameter aktual akan disalin ke parameter formal.
- Dengan cara ini nilai parameter aktual tidak bisa berubah sekalipun nilai parameter formal berubah.

CARA MELEWATKAN PARAMETER (CONT.)

- Perhatikan contoh Program_4 berikut :

```
1.  #include <stdio.h>
2.  void tukar (int x, int y);
3.  main()
4.  {
5.     int a,b;
6.     a = 88;
7.     b = 77;
8.     printf("Nilai sebelum pemanggilan fungsi\n");
9.     printf("a = %d b = %d\n\n",a,b);
10.    tukar(a,b);
11.   printf("Nilai sesudah pemanggilan fungsi\n");
12.   printf("a = %d b = %d\n\n",a,b);
13. }
14.
```

CARA MELEWATKAN PARAMETER (CONT.)

```
15. void tukar (int x, int y)
16. {
17.    int z;
18.    z = x;
19.    x = y;
20.    y = z;
21.    printf("Nilai diakhir fungsi tukar()\n");
22.    printf("x = %d y = %d\n\n",x,y);
23. }
```

hasil eksekusi :
c> Program_6
Nilai sebelum pemanggilan fungsi
a = 88 b = 77
Nilai diakhir fungsi tukar()
x = 77 y = 88
Nilai sesudah pemanggilan fungsi
a = 88 b = 77

CARA MELEWATKAN PARAMETER (CONT.)

- Dari contoh Program_4 terlihat bahwa hasil pemanggilan fungsi tukar( ), variabel a dan b (yang dilewatkan ke fungsi tukar ( ) tidak berubah, walaupun pada fungsi tukar ( ) telah terjadi penukaran antara parameter x dan y.
- Hal ini terjadi karena **x hanyalah salinan dari a dan y salinan dari b, serta definisi fungsi tidak mengembalikan nilai (diawali dengan void dan tidak ada instruksi return( )).**

CARA MELEWATKAN PARAMETER (CONT.)

- Pemanggilan dengan **referensi merupakan upaya untuk melewatkan alamat dari suatu variabel ke dalam fungsi.**
- Cara ini dapat digunakan untuk mengubah isi suatu variabel di luar fungsi dengan pelaksanaan pengubahan dilakukan di dalam fungsi.

CARA MELEWATKAN PARAMETER DENGAN REFERENSI

Perhatikan contoh Program_5 berikut :

```
1.  #include <stdio.h>
2.  void tukar (int *px, int *py);
3.  main()
4.  {
5.     int a,b;
6.     a = 88;
7.     b = 77;
8.     printf("Nilai sebelum pemanggilan fungsi\n");
9.     printf("a = %d b = %d\n\n",a,b);
10.    tukar(&a,&b); /* parameter alamat a dan alamat b */
11.    printf("Nilai sesudah pemanggilan fungsi\n");
12.    printf("a = %d b = %d\n\n",a,b);
13. }
```

CARA MELEWATKAN PARAMETER DENGAN REFERENSI (CONT.)

```
void tukar (int *px, int *py)
1.  {
2.     int z;
3.     z = *px;
4.     *px = *py;
5.     *py = z;
6.     printf("Nilai diakhir fungsi tukar()\n");
7.     printf("*px = %d *py = %d\n\n",*px,*py);
8.  }
```

hasil eksekusi :

```
c> Program_7
Nilai sebelum pemanggilan fungsi
a = 88 b = 77
Nilai diakhir fungsi tukar()
*px = 77 *py = 88
Nilai sesudah pemanggilan fungsi
a = 77 b = 88
```

TUGAS

- Buatlah flowchart dan program untuk mencetak nama dan alamat anda dengan prinsip fungsi yang menggunakan parameter dan tanpa parameter.
- Buatlah flowchart dan program untuk menghitung luas segitiga dengan prinsip fungsi menggunakan *return* dan tanpa *return*.

TUGAS (CONT.)

- Buatlah flowchart dan program untuk menghitung luas lingkaran yang mengimplementasikan prinsip parameter pemanggilan nilai, dimana inisialisasi sebelumnya diubah didalam fungsi.
- Seperti soal sebelumnya hanya menggunakan parameter pemanggilan referensi.